

Table of Contents

1. Contesco Theme Setup.....	2
1.1. Overview	2
1.2. System Requirements	2
1.3. Setting Up Contesco	2
1.3.1. Step 1: Uninstall Existing Theme	2
1.3.2. Step 2: Upload Theme	2
1.3.3. Step 3: Modify Theme	2
1.3.4. Step 4: Flush Cache	3
1.3.5. Step 5: Update Theme	6
1.3.6. Step 6: Deploy Theme	6
1.3.7. Step 7: Configure Theme	6
1.3.8. Step 8: Modify Theme	6
1.3.9. Step 9: Backup Theme	6
1.3.10. Step 10: Deploy and Activate the Theme	7
1.3.11. Step 11: Configure the Deployed Theme	7
1.4. Conclusion	7

1. Contesco Theme Setup

1.1. Overview

This document outlines the steps for setting up and deploying the Contesco Theme project. The Platform support team will provide you the source code (Contesco_theme.zip) along with detailed instructions to help you customize and deploy the theme. Comments in the source code files will guide you through the required changes.

1.2. System Requirements

- Maven Version: 3.6.3 and higher versions
- Java Version: 1.8 and higher versions

1.3. Setting Up Contesco Theme Project

The following are the steps to configure and deploy the Contesco theme.

1.3.1. Step 1: Update pom.xml Configuration

1. Open the pom.xml file.
2. Update the configuration with the following values:

<code><groupId></code>		<code>groupId></code>
<code><artifactId></code>		
<code><name></code>		
3. Scroll to the <code><finalName></code> tag and update the value to:		
<code><finalName></code>		

Note: The "contesco" used here is an example. You can rename it as per your project's needs.

The Contesco source code is available in the Contesco_theme.zip file, which contains all the necessary files and instructions.

1.3.2. Step 2: Update bnd.bnd File

Modify the bnd.bnd file to include the following:

- **Bundle-Name:** Contesco Theme
- **Web-ContextPath:** /contesco-theme
- **Bundle-SymbolicName:** com.platform.portal.theme.contesco

1.3.3. Step 3: Modify Theme Configuration Files

1. Navigate to the `theme` directory.
2. Update the `theme.properties` file.

<code>long-content-width=1000</code>		
<code>module-name=ContescoTheme</code>		
<code>name=ContescoTheme</code>		

3. Update the **liferay-look-and-feel.xml** file:

- Modify the **<theme>** tag to the following:

```
<theme id="Contesco" name="Contesco" css-class="Contesco" />
<setting name="Contesco" type="checkbox" value="true" />
<setting name="Contesco" type="checkbox" value="true" />
</setting>
```

Note: Additional custom settings can be implemented similarly to header and footer settings.

- Define different color schemes for the theme (optional):

```
<setting name="Contesco" type="checkbox" value="true" />
<color-scheme name="Contesco" css-class="Contesco" />
</color-scheme>
</setting>

id - Unique identifier for the theme.
name - Name of the theme.
css-class - CSS class to be applied to the theme when this scheme is active.
```

Note: Just like color schemes, we can also configure layouts, permissions, and CSS/JS resources. (Reference: <http://www.liferay.com/doc/liferay-portal-6-0/html/contesco-theme-setup.html>)

4. Retrieving Theme Settings

- Include the theme settings in the theme's JSPs:

```
<#assign themeSettings = portletDisplay.themeSettings />
</#assign>
<#assign themeSettings = portletDisplay.themeSettings />
</#assign>

<#include "themeSettings.ftl" />
```

1.3.4. Step 4: FreeMarker (FTL)

1.3.4.1. Objects

ThemeDisplay:

Provides context information about the current page, layout, and user.

In the

```
<#include "themeSettings.ftl" />
```

It offers access to the current page, layout, CSS, images, JavaScript, current language, and even user details.

PortletDisplay:

This object (from portletDisplay) gives details about the current portlet

- instance—its configuration, title, and state. It is similarly assigned as

```
<#assign instance = ... />
```

making its pro

Company:

The **company** object is used to access company-specific details.

is used to access company-specific

Examples in th

```
<#assign company = ... />
<#assign company.getName() />
```

It also provide

User:

The **user** object holds the current user's data.

Later in the file, properties like the full name, email, greeting, and more are fetched using methods

s(

)) ensure that user properties are only assigned if they

1.3.4.2. Variables

FreeMarker uses the **<#assign>** directive to create and initialize variables. This allows you to store reusable data throughout your template.

Exam

```
<#assign ... />
```

1.3.4

FreeMarker uses the **<#if>** and **<#else>** directives.

```
<#if ... />
<#else ... />
</#if>
```

This depends an extra CSS class.

1.3.4

Although loops in FreeMarker are implemented using

```
<#list ... />
```

This iterates over a collection (e.g., navigation items) and outputs each item.

1.3.4.5. Accessing Web Content in FreeMarker

While the `init.ftl` file mainly sets up the theme's environment, the same principles apply when accessing web content:

Web Content Structures:

When you define a web content structure in Liferay, you set up fields (like title, body, etc.) that become variables in your FreeMarker template.

Example:

```
<h1>${title}</h1>
<div>${body}</div>
```

1.3.4.6. Including External JavaScript Files

Themes typically include external JavaScript files by dynamically generating their URLs:

Generating

In the `init.ftl`

```
<#assign
theme
```

This creates

```
<script
```

This ensures

1.3.4.7. Including External CSS Files

Generating

The CSS for

```
<#assign
```

```
<link
/>
```

This method

browser.

1.3.4.8. Including External Images

1. C

2. In

E

```
<link
href=
.css
rel=
```

3. In

E

```
<script
src=
es.m
```

ect as below.

g.

```
ss()>
```

```
s_main_file}"
```

oped for use in the

```
ataTables.min
```

```
able/dataTabl
```

1.3.5. Step 5: Update the CSS

1. Open the `src/main/webapp/css/` directory to customize the login styles.
2. Modify the `login.css` file.
3. Update the `login.css` file to customize the login styles.

For example, the code for the `login.css` file is as follows:

1.3.6. Step 6: Design the Header

1. Open the `src/main/webapp/css/` directory to design the header.
 2. Update the `header.css` file.
- For instance, the base code for the header is in the `_header.scss` file.

1.3.7. Step 7: Customize the Navigation

1. Edit the `src/main/webapp/css/` directory to design the side navigation.
2. Update the `navigation.css` file.

As an example, the base code for the header (orange) in the `_navigation.scss` file.

1.3.7.1. Including Plugins

1. Create a directory named **lib** (or **library**) inside the **webapps** folder.
2. Include the CSS by adding this code to your HTML:

```
<link href="lib/datatables.min.css" rel="stylesheet">
3.
<script src="lib/datatables.min.js"></script>
```

1.3.8. Step 8: Modify the Navbar and Scrollbar

1. Customize the navbar by adding the `src/main/webapp/css/` directory.
2. Modify the scrollbar by adding the `src/main/webapp/css/` directory.
3. Add header logos and icons by adding the `src/main/webapp/images/` directory.

1.3.9. Step 9: Build and Deploy Theme

After completing all design changes, build the WAR file by running the following Maven command from the project directory:

mvn clean install

Once the build completes, the WAR file will be located in the `target/` directory within the project folder:

Path: `Sample-project/target/`

1.3.10. Step 10: Deploy the WAR File to the Server

1. Deploy the generated WAR file to your server.
2. Once the WAR file is deployed, the theme will appear in the Platform Control Panel.

1.3.11. Step 11:

1. Log in to the Platform.
2. In the Platform title bar, the theme will appear on the left side of the page.
3. Click and expand the theme.
4. In the Settings (next to the theme), you can change the theme's Feel.
5. Scroll to the bottom of the page.
6. Select the newly deployed theme as the default theme.

Optional: If you have customized the login page, repeat the steps above to apply the same theme configuration for the login page.

1.4. Conclusion

By following the steps mentioned in this documentation, you will be able to successfully create, customize, and deploy the **Contesco** theme. The project is designed to be flexible, allowing you to modify key elements such as login structure, header, navigation, and overall theme style to match your requirements.